

DSA0601-DATA HANDLING AND VISUALIZATION

SET 1: MONTHLY SALES DATA

Name: Mohamed nazeer S

reg no:192424084

Dataset: Monthly Sales

Month	Sales (in \$)
January	15,000
February	18,000
March	22,000
April	20,000
May	23,000

QUESTION 1 – LINE CHART: MONTHLY SALES

AIM

To create a line chart for visualizing the monthly sales trend from January to May and identify changes in sales over time.

ALGORITHM

1. Import the Matplotlib library.
2. Store the month names and corresponding sales values in lists.
3. Create a line chart using the plot() function.
4. Add markers to indicate individual monthly sales values.
5. Label the X-axis as Month and Y-axis as Sales (\$).
6. Add a suitable chart title and grid.
7. Display the line chart.

PROCEDURE

Open Google Colab or a Python environment. Create a new code cell, copy the program below and execute it. The monthly sales values are entered directly in the program. The generated line chart displays the sales trend from January to May.

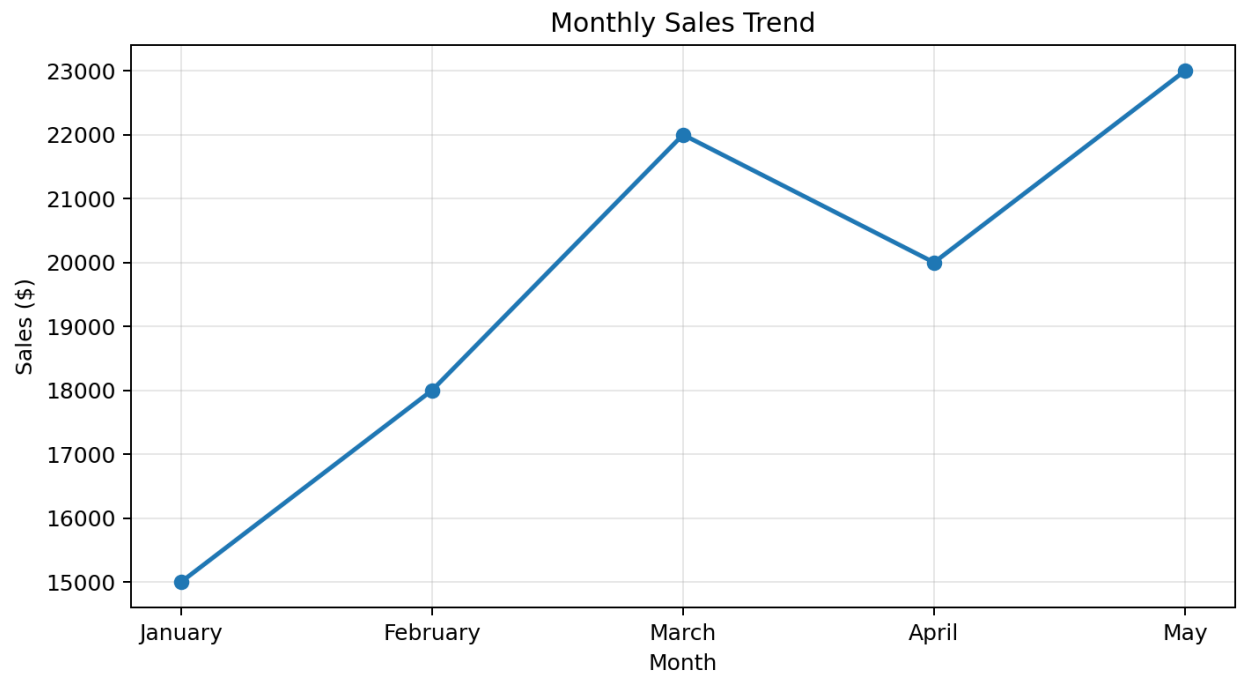
CODE

```
import matplotlib.pyplot as plt
```

```
months = ["January", "February", "March", "April", "May"]
sales = [15000, 18000, 22000, 20000, 23000]

plt.figure(figsize=(8, 5))
plt.plot(months, sales, marker="o", linewidth=2)
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.title("Monthly Sales Trend")
plt.grid(True)
plt.show()
```

OUTPUT SCREENSHOT



RESULT

Thus, the line chart for monthly sales was created successfully. The chart shows an overall increasing sales trend from \$15,000 in January to \$23,000 in May, with a small decrease in April.

QUESTION 2 – BAR CHART: TOP-SELLING PRODUCTS

AIM

To create a bar chart for comparing the top-selling products and identifying the product with the highest and lowest sales.

ALGORITHM

1. Import the Matplotlib library.
2. Store product names and units sold in separate lists.
3. Create a bar chart using the bar() function.
4. Place product names on the X-axis.
5. Place units sold on the Y-axis.
6. Add axis labels and a suitable chart title.
7. Display the bar chart.

PROCEDURE

Open Google Colab or a Python environment. Enter the sample top-selling product dataset directly in the program. Run the code to generate the bar chart. Compare the height of the bars to identify the highest and lowest selling products.

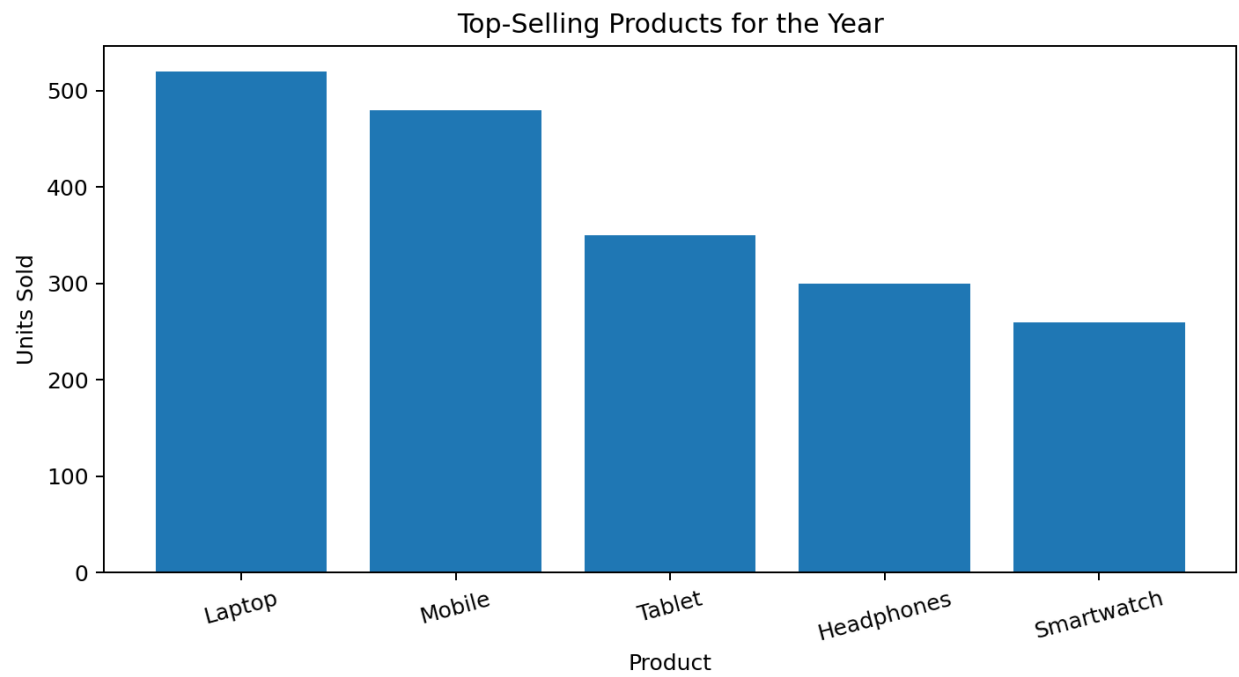
CODE

```
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet",
            "Headphones", "Smartwatch"]
units_sold = [520, 480, 350, 300, 260]

plt.figure(figsize=(8, 5))
plt.bar(products, units_sold)
plt.xlabel("Product")
plt.ylabel("Units Sold")
plt.title("Top-Selling Products for the Year")
plt.xticks(rotation=15)
plt.show()
```

OUTPUT SCREENSHOT



RESULT

Thus, the bar chart for top-selling products was created successfully. From the sample data, Laptop is the top-selling product with 520 units, while Smartwatch has the lowest sales among the listed products with 260 units.

QUESTION 3 – SCATTER PLOT: ADVERTISING BUDGET VS SALES

AIM

To develop a scatter plot to study the relationship between advertising budget and monthly sales and interpret the pattern between the two variables.

ALGORITHM

1. Import the Matplotlib library.
2. Store advertising budget and monthly sales values in lists.
3. Create a scatter plot using the scatter() function.
4. Plot Advertising Budget on the X-axis.
5. Plot Monthly Sales on the Y-axis.
6. Add labels, title and grid to the chart.
7. Display the scatter plot and observe the direction of the points.
8. Interpret the relationship between advertising budget and sales.

PROCEDURE

Open Google Colab or a Python environment. Enter the advertising budget and monthly sales values directly in the code. Execute the program. Observe the pattern of plotted points. An upward movement of points indicates a positive relationship between advertising expenditure and sales.

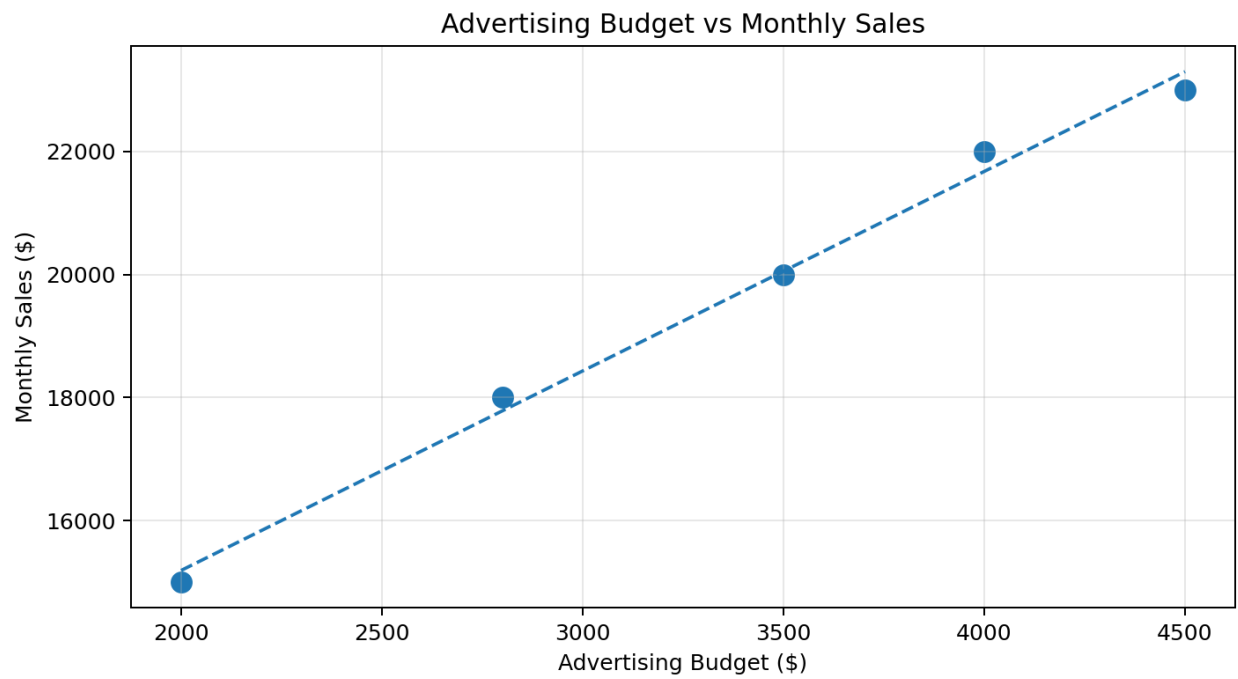
CODE

```
import matplotlib.pyplot as plt

advertising_budget = [2000, 2800, 4000, 3500, 4500]
monthly_sales = [15000, 18000, 22000, 20000, 23000]

plt.figure(figsize=(8, 5))
plt.scatter(advertising_budget, monthly_sales, s=80)
plt.xlabel("Advertising Budget ($)")
plt.ylabel("Monthly Sales ($)")
plt.title("Advertising Budget vs Monthly Sales")
plt.grid(True)
plt.show()
```

OUTPUT SCREENSHOT



RESULT

Thus, the scatter plot was developed successfully. The points show a positive relationship between advertising budget and monthly sales. In general, higher advertising expenditure is associated with higher sales. This suggests that advertising may contribute to sales growth, although the scatter plot alone does not prove that advertising is the only factor affecting sales.

QUESTION 4 – INTERACTIVE SALES DASHBOARD

AIM

To build an interactive sales dashboard that combines the monthly sales line chart and top-selling products bar chart, allowing users to explore sales information interactively.

ALGORITHM

1. Import Pandas, Plotly Express and Dash libraries.
2. Create the monthly sales DataFrame.
3. Create the product sales DataFrame.
4. Generate an interactive Plotly line chart for monthly sales.
5. Generate an interactive Plotly bar chart for top-selling products.
6. Create a Dash web application.
7. Add a dashboard heading and both charts to the application layout.
8. Run the Dash application.
9. Use hover, zoom and chart controls to interactively explore the sales data.

PROCEDURE

In Google Colab, first install Dash using `!pip install dash`. Copy the complete dashboard program into a code cell and execute it. Open the dashboard output and move the pointer over chart elements to view values. Plotly controls can be used for zooming and exploring the charts.

CODE

```
# Install once in Google Colab:
# !pip install dash

import pandas as pd
import plotly.express as px
from dash import Dash, dcc, html

sales_df = pd.DataFrame({
    "Month": ["January", "February", "March",
              "April", "May"],
    "Sales": [15000, 18000, 22000, 20000, 23000]
})

product_df = pd.DataFrame({
    "Product": ["Laptop", "Mobile", "Tablet",
                "Headphones", "Smartwatch"],
    "Units_Sold": [520, 480, 350, 300, 260]
})

line_fig = px.line(
    sales_df, x="Month", y="Sales", markers=True,
    title="Monthly Sales Trend"
)
```

```

bar_fig = px.bar(
    product_df, x="Product", y="Units_Sold",
    title="Top-Selling Products for the Year"
)

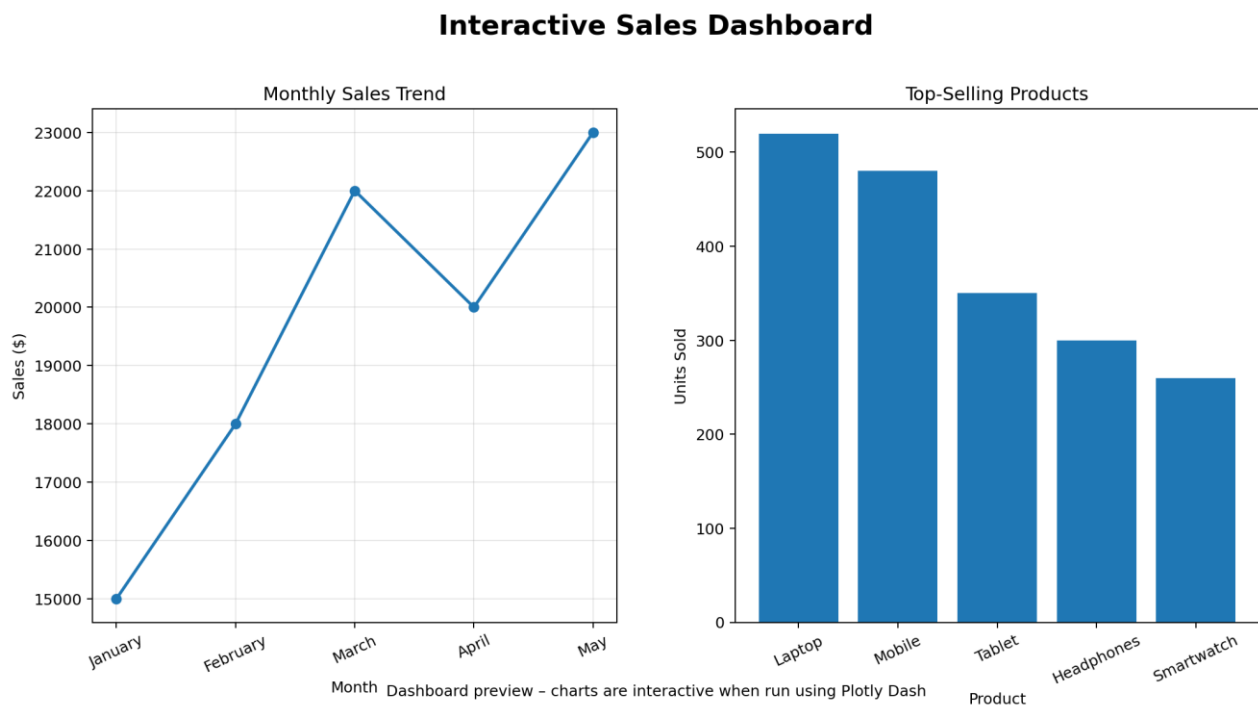
app = Dash(__name__)

app.layout = html.Div([
    html.H1("Interactive Sales Dashboard",
        style={"textAlign": "center"}),
    dcc.Graph(figure=line_fig),
    dcc.Graph(figure=bar_fig)
])

if __name__ == "__main__":
    app.run(debug=False)

```

OUTPUT SCREENSHOT



RESULT

Thus, an interactive sales dashboard combining the monthly sales line chart and top-selling products bar chart was developed successfully using Plotly Dash. Users can interact with the charts through hover information, zooming, panning and other Plotly chart controls.